

Reviewer Pack — Algebraic K-Theory Rank and Circuit Monotone Width Inequalit...

Entry 10dcc97bae56 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 08:58:30 UTC

Algebraic K-Theory Rank and Circuit Monotone Width Inequality

Entry ID: 10dcc97bae56 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 08:58:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every Boolean circuit C with n inputs, the rank of its associated module in Algebraic K-Theory ($r_K(C)$) is linearly correlated with its monotone width $w_{\text{mon}}(C)$, such that $r_K(C) = \Theta(w_{\text{mon}}(C))$. Furthermore, for all circuits C , if $r_K(C) > 2 * w_{\text{mon}}(C)$, then C contains an AND-gate.

Rationale (proposer's reasoning):

Algebraic K-Theory provides a framework to study the structure of rings and modules over them. By associating each circuit with an algebraic module, we can potentially uncover deeper connections between its algebraic properties and complexity measures like monotone width, which is a fundamental measure in circuit complexity theory.

Taxonomy category: ALGEBRAIC_K_THEORY_MONOTONE_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 94f366dcb059cc2e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture on Algebraic K-Theory rank and circuit monotone width inequality, support is indicated by a correlation coefficient of $r_K(C)$ with $w_{\text{mon}}(C) \geq 0.8$ for at least 80% of seeds, AND-gates detected in circuits where $r_K(C) > 2 * w_{\text{mon}}(C)$, and no seed producing $|r_K(C) - w_{\text{mon}}(C)| > 3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Algebraic K-Theory" AND "Boolean Circuit Complexity" AND monotone width" - "r_K(C)" AND " $\theta(w_{\text{mon}}(C))$ " AND "circuit complexity" - AND-gate AND "Algebraic K-Theory" AND "monotone width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return [random.choice([0, 1])]
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return [random.choice([0, 1]) for _ in range(n)]

    def monotone_width(circuit):
        if len(circuit) == 1:
            return 1
        else:
            left_width = monotone_width(circuit[:len(circuit)//2])
            right_width = monotone_width(circuit[len(circuit)//2:])
            return max(left_width, right_width)

    def algebraic_k_theory_rank(circuit):
        if len(circuit) == 1:
            return 1
        else:
            left_rank = algebraic_k_theory_rank(circuit[:len(circuit)//2])
            right_rank = algebraic_k_theory_rank(circuit[len(circuit)//2:])
            return max(left_rank, right_rank) + 1

    n_max = 40
    instances_tested = 30
    metric_values = []
```

```

conjecture_holds = True
counterexample = ""

for _ in range(instances_tested):
    n = random.randint(5, n_max)
    circuit = generate_circuit(n)
    r_K = algebraic_k_theory_rank(circuit)
    w_mon = monotone_width(circuit)

    if abs(r_K - w_mon) > 3:
        counterexample = f"r_K({n})={r_K}, w_mon({n})={w_mon}"
        conjecture_holds = False
        break

    metric_values.append((r_K, w_mon))

mean_r_K = sum(x[0] for x in metric_values) / len(metric_values)
mean_w_mon = sum(x[1] for x in metric_values) / len(metric_values)
correlation_coefficient = (sum((x[0] - mean_r_K) * (x[1] - mean_w_mon) for x in metric_values) /
                           math.sqrt(sum((x[0] - mean_r_K)**2 for x in metric_values)) *
                           math.sqrt(sum((x[1] - mean_w_mon)**2 for x in metric_values)))

return {
    "metric_name": "Algebraic K-Theory Rank vs Monotone Width",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ad3e126.py", line 85, in <module>
    result = run_trial(seed)
             ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3ad3e126.py", line 64, in run_trial
    mean_r_K = sum(x[0] for x in metric_values) / len(metric_values)
               ~~~~~^
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it was unable to complete the required computations and verify the conjecture. | next: Re-run the test with appropriate error handling to ensure that it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14426
2	propose	ollama_remote	glm4:latest	0	0	16900
3	preregistration	ollama_remote	glm4:latest	0	0	10013
4	novelty	ollama_remote	glm4:latest	0	0	12252
5	novelty	ollama_remote	glm4:latest	0	0	13884
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14926
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12177
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11000
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10061
10	judge	ollama_remote	glm4:latest	0	0	32723

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148362 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/10dcc97bae56.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/10dcc97bae56.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/10dcc97bae56.tar.gz` (if generated)